

EEEP MARIA CAVALCANTE COSTA

EQUIPE 2

FRANCISCA VITÓRIA QUEIROZ OLIVEIRA

MIKAEL DE MELO BRAZ

MATEUS DE SOUSA OLIVEIRA

CLARISSE DE SOUSA SÁ

JOÃO VICTOR MATEUS DE ASSIS

RELATÓRIO

SISTEMA DE BIBLIOTECAS

PROF: CARLOS ALEXANDRE

COMPONENTE: PROGRAMAÇÃO ORIENTADA A OBJETOS

CURSO: 2 INFORMÁTICA

DATA DE ENTREGA: 21/06/2026

QUIXADÁ - CE

SISTEMA DE BIBLIOTECAS

O sistema tem como temática, realizar o cadastro e gerenciamento de livros, usuários e empréstimos, permitindo operações básicas através da **Linguagem Java** em relação à disciplina de **Programação Orientada a Objetos** com o uso de classes e objetos construtores. O objetivo é que a **Biblioteca** mantenha uma coleção (lista ou vetor) de objetos **Livro** e execute as operações de cadastro, busca, empréstimo e devolução alterando o atributo **status** de cada livro.

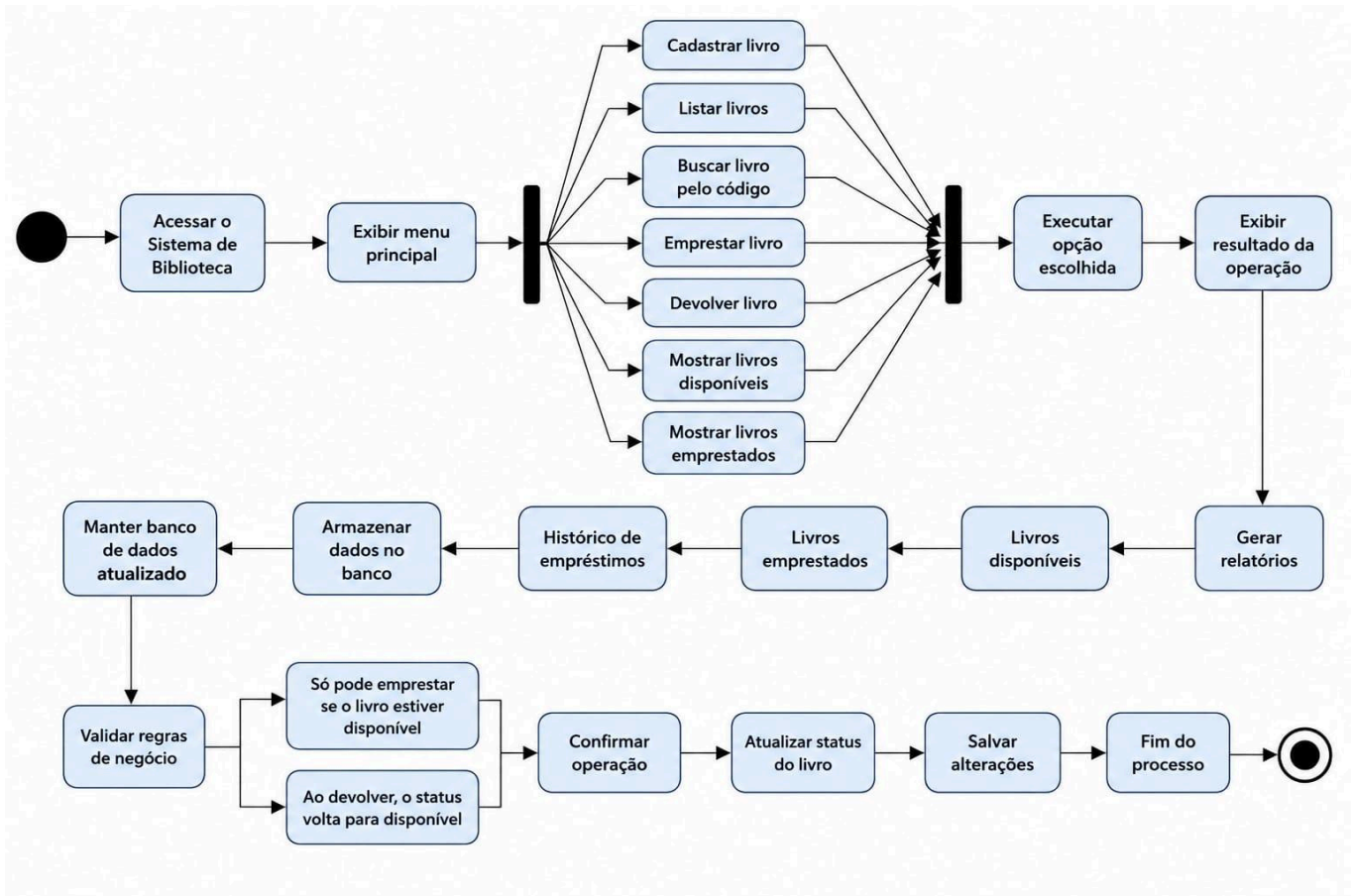
★ REQUISITOS FUNCIONAIS

- Cadastrar livro
- Listar livros
- Buscar livro pelo código
- Emprestar livro
- Devolver livro
- Mostrar livros disponíveis
- Mostrar livros emprestados
- Sair

★ CONCEITOS DE POO UTILIZADOS

- Classes
- Objetos
- Métodos
- Construtores
- Encapsulamento
- Vetores
- Herança
- Polimorfismo

DIAGRAMA DE CLASSES



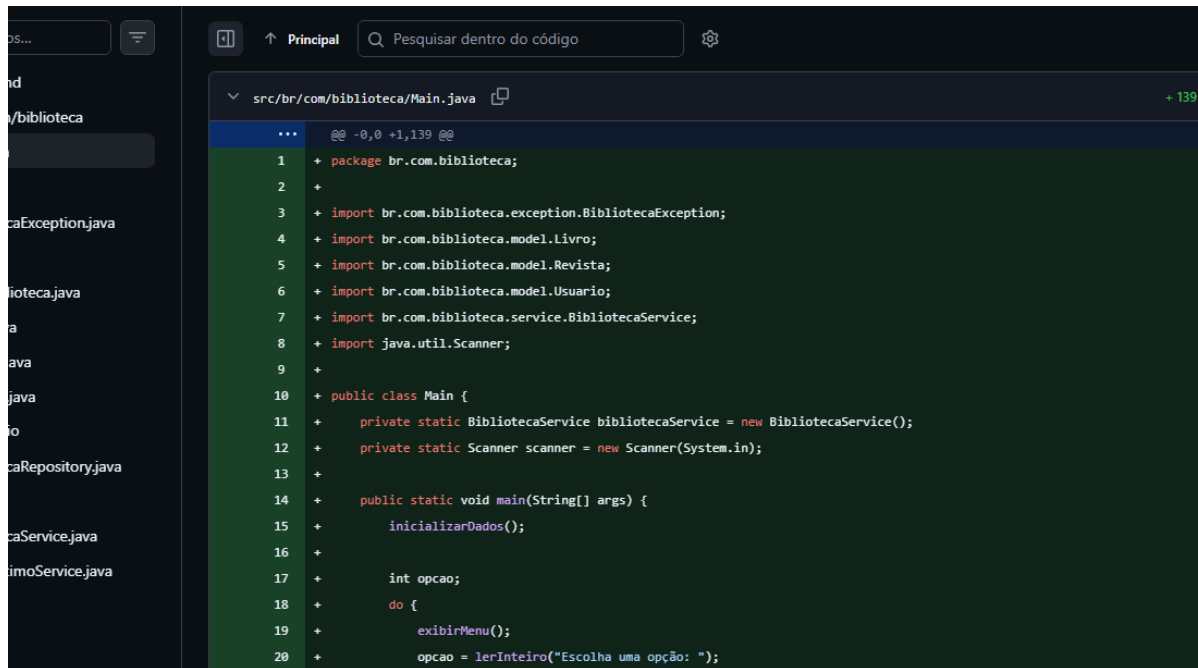
ESTRUTURA DO PROJETO

Projeto Biblioteca

☐ Main JAVA

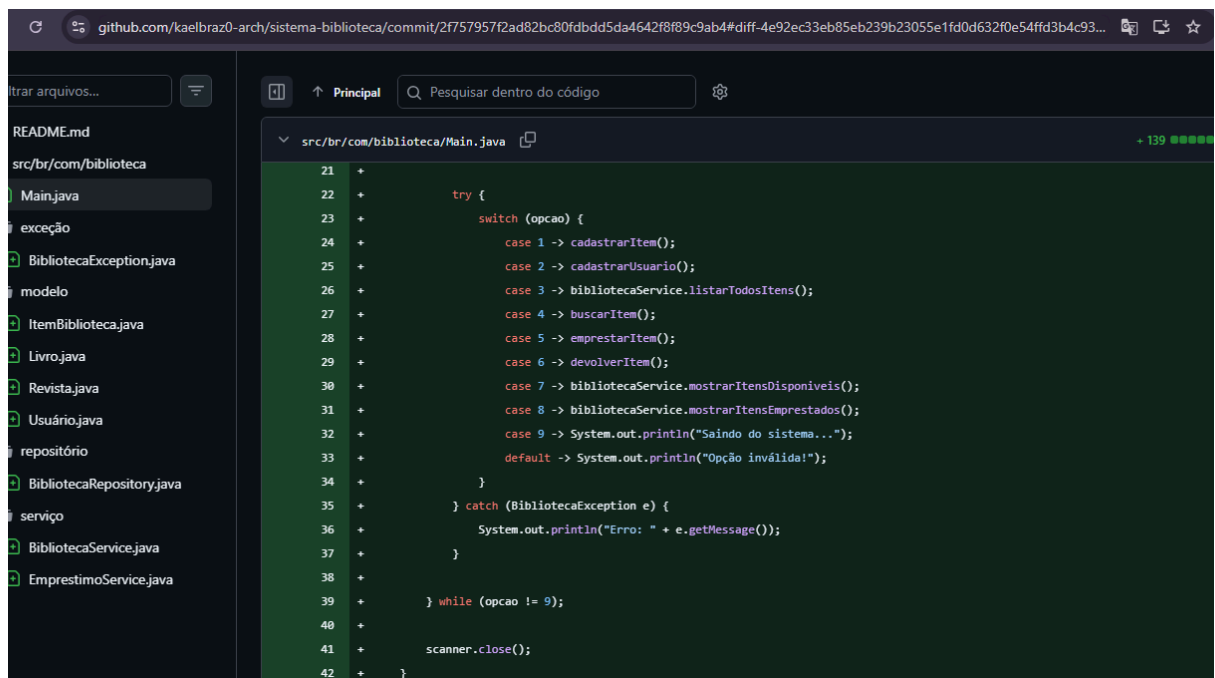
Responsabilidade: Executar o sistema

Código fonte:



The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'biblioteca' with a 'src/br/com/biblioteca/' directory. The code editor displays the 'Main.java' file, which contains the following code:

```
1 + package br.com.biblioteca;
2 +
3 + import br.com.biblioteca.exception.BibliotecaException;
4 + import br.com.biblioteca.model.Livro;
5 + import br.com.biblioteca.model.Revista;
6 + import br.com.biblioteca.model.Usuario;
7 + import br.com.biblioteca.service.BibliotecaService;
8 + import java.util.Scanner;
9 +
10 + public class Main {
11 +     private static BibliotecaService bibliotecaService = new BibliotecaService();
12 +     private static Scanner scanner = new Scanner(System.in);
13 +
14 +     public static void main(String[] args) {
15 +         inicializarDados();
16 +
17 +         int opcao;
18 +         do {
19 +             exibirMenu();
20 +             opcao = lerInteiro("Escolha uma opção: ");
```



The screenshot shows a web browser displaying the GitHub repository for the 'sistema-biblioteca' project. The URL is 'github.com/kaelbraz0-arch/sistema-biblioteca/commit/2f757957f2ad82bc80fdbdd5da4642f8f89c9ab4#diff-4e92ec33eb85eb239b23055e1fd0d632f0e54ffd3b4c93...'. The file explorer on the left shows the project structure, including 'README.md', 'src/br/com/biblioteca/', and 'Main.java'. The code editor displays the 'Main.java' file, which contains the following code:

```
21 +
22 +     try {
23 +         switch (opcao) {
24 +             case 1 -> cadastrarItem();
25 +             case 2 -> cadastrarUsuario();
26 +             case 3 -> bibliotecaService.listarTodosItens();
27 +             case 4 -> buscarItem();
28 +             case 5 -> emprestarItem();
29 +             case 6 -> devolverItem();
30 +             case 7 -> bibliotecaService.mostrarItensDisponiveis();
31 +             case 8 -> bibliotecaService.mostrarItensEmprestados();
32 +             case 9 -> System.out.println("Saindo do sistema...");
33 +             default -> System.out.println("Opção inválida!");
34 +         }
35 +     } catch (BibliotecaException e) {
36 +         System.out.println("Erro: " + e.getMessage());
37 +     }
38 +
39 +     } while (opcao != 9);
40 +
41 +     scanner.close();
42 + }
```

github.com/kaelbraz0-arch/sistema-biblioteca/commit/2f757957f2ad82bc80fdbdd5da4642f8f89c9ab4#diff-4e92ec33eb85eb239b23055e1fd0d632f0e54ffd3b4c93...

Principal Pesquisar dentro do código

src/br/com/biblioteca/Main.java +139

```
44 + private static void exibirMenu() {
45 +     System.out.println("\n== SISTEMA DE BIBLIOTECA ==");
46 +     System.out.println("1. Cadastrar Item (Livro/Revista)");
47 +     System.out.println("2. Cadastrar Usuário");
48 +     System.out.println("3. Listar Todos os Itens");
49 +     System.out.println("4. Buscar Item por Código");
50 +     System.out.println("5. Emprestar Item");
51 +     System.out.println("6. Devolver Item");
52 +     System.out.println("7. Mostrar Itens Disponíveis");
53 +     System.out.println("8. Mostrar Itens Emprestados");
54 +     System.out.println("9. Sair");
55 +     System.out.print("Opção: ");
56 + }
57 +
58 + private static void cadastrarItem() {
59 +     System.out.println("\n--- Cadastro de Item ---");
60 +     System.out.println("1. Livro");
61 +     System.out.println("2. Revista");
62 +     int tipo = lerInteiro("Tipo de item: ");
63 +
64 +     System.out.print("Código: ");
```

github.com/kaelbraz0-arch/sistema-biblioteca/commit/2f757957f2ad82bc80fdbdd5da4642f8f89c9ab4#diff-4e92ec33eb85eb239b23055e1fd0d632f0e54ffd3b4c93...

Principal Pesquisar dentro do código

src/br/com/biblioteca/Main.java +139

```
65 + String codigo = scanner.nextLine();
66 + System.out.print("Título: ");
67 + String titulo = scanner.nextLine();
68 +
69 + if (tipo == 1) {
70 +     System.out.print("Autor: ");
71 +     String autor = scanner.nextLine();
72 +     int ano = lerInteiro("Ano de publicação: ");
73 +     System.out.print("ISBN: ");
74 +     String isbn = scanner.nextLine();
75 +
76 +     bibliotecaService.cadastrarItem(new Livro(codigo, titulo, autor, ano, isbn));
77 + } else if (tipo == 2) {
78 +     System.out.print("Edição: ");
79 +     String edicao = scanner.nextLine();
80 +     System.out.print("Mês de referência: ");
81 +     String mes = scanner.nextLine();
82 +
83 +     bibliotecaService.cadastrarItem(new Revista(codigo, titulo, edicao, mes));
84 + } else {
85 +     System.out.println("Tipo inválido!");
86 + }
```

github.com/kaelbraz0-arch/sistema-biblioteca/commit/2f757957f2ad82bc80fdbdd5da4642f8f89c9ab4#diff-4e92ec33eb85eb239b23055e1fd0d632f0e54ffd3b4c93...

src/br/com/biblioteca

Main.java

```
86 + }
87 + }
88 +
89 + private static void cadastrarUsuario() {
90 +     System.out.println("\n--- Cadastro de Usuário ---");
91 +     System.out.print("ID do usuário: ");
92 +     String id = scanner.nextLine();
93 +     System.out.print("Nome completo: ");
94 +     String nome = scanner.nextLine();
95 +     System.out.print("Email: ");
96 +     String email = scanner.nextLine();
97 +
98 +     bibliotecaService.cadastrarUsuario(new Usuario(id, nome, email));
99 + }
100 +
101 + private static void buscarItem() {
102 +     System.out.print("\nDigite o código do item: ");
103 +     String codigo = scanner.nextLine();
104 +     bibliotecaService.buscarItem(codigo);
105 + }
106 +
107 + private static void emprestarItem() throws BibliotecaException {
```

github.com/kaelbraz0-arch/sistema-biblioteca/commit/2f757957f2ad82bc80fdbdd5da4642f8f89c9ab4#diff-4e92ec33eb85eb239b23055e1fd0d632f0e54ffd3b4c93...

src/br/com/biblioteca

Main.java

```
107 + private static void emprestarItem() throws BibliotecaException {
108 +     System.out.println("\n--- Empréstimo ---");
109 +     System.out.print("Código do item: ");
110 +     String codigoItem = scanner.nextLine();
111 +     System.out.print("ID do usuário: ");
112 +     String idUsuario = scanner.nextLine();
113 +
114 +     bibliotecaService.emprestarItem(codigoItem, idUsuario);
115 + }
116 +
117 + private static void devolverItem() throws BibliotecaException {
118 +     System.out.println("\n--- Devolução ---");
119 +     System.out.print("Código do item: ");
120 +     String codigoItem = scanner.nextLine();
121 +
122 +     bibliotecaService.devolverItem(codigoItem);
123 + }
124 +
125 + private static int lerInteiro(String mensagem) {
126 +     System.out.print(mensagem);
127 +     int valor = scanner.nextInt();
128 +     scanner.nextLine();
```

github.com/kaelbraz0-arch/sistema-biblioteca/commit/2f757957f2ad82bc80fdbdd5da4642f8f89c9ab4#diff-4e92ec33eb85eb239b23055e1fd0d632f0e54ffd3b4c93...

src/br/com/biblioteca

Main.java

```
128 +     scanner.nextLine();
129 +     return valor;
130 + }
131 +
132 + private static void inicializarDados() {
133 +     bibliotecaService.cadastrarItem(new Livro("L001", "Clean Code", "Robert Martin", 2008, "9780132350884"));
134 +     bibliotecaService.cadastrarItem(new Livro("L002", "Design Patterns", "Erich Gamma", 1994, "9780261633618"));
135 +     bibliotecaService.cadastrarItem(new Revista("R001", "Veja", "Ed. 2756", "Janeiro/2024"));
136 +     bibliotecaService.cadastrarUsuario(new Usuario("U001", "João Silva", "joao@email.com"));
137 +     bibliotecaService.cadastrarUsuario(new Usuario("U002", "Maria Santos", "maria@email.com"));
138 + }
139 + }
```

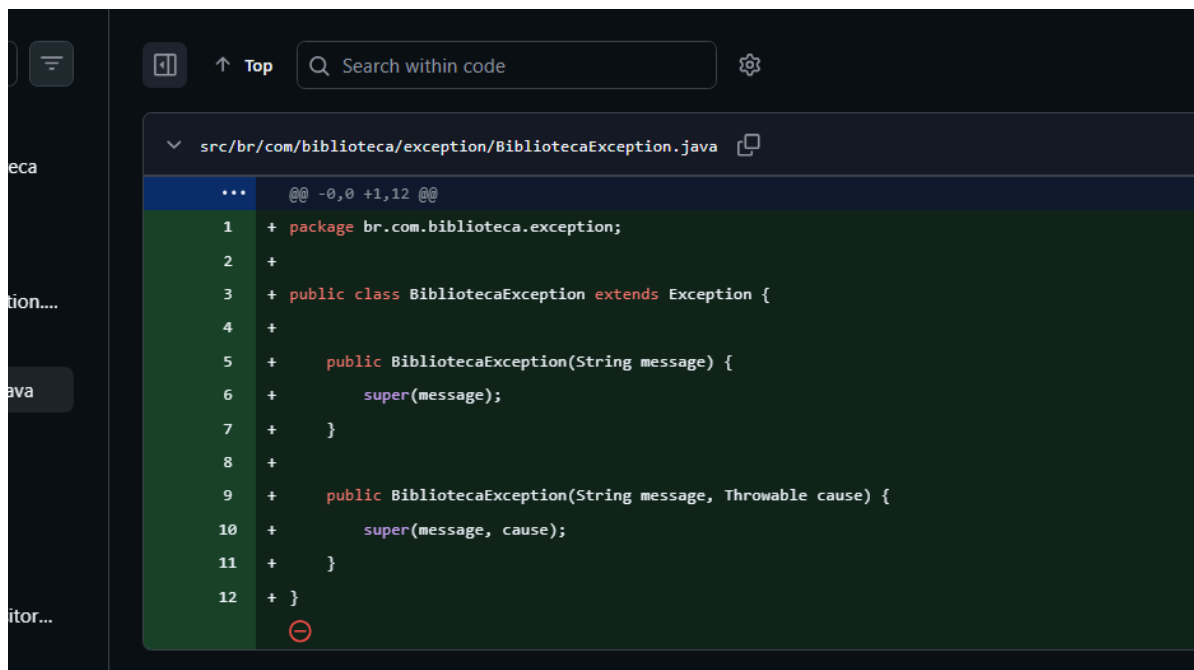
☐ Biblioteca Exception

Responsabilidade: Criar exceções próprias da biblioteca e informar mensagens de erro específicas para problemas como: “Livro não encontrado” e etc.

Atributos: Nenhum, porém ela herda atributos da classe Exception, como: message e cause

Métodos: BibliotecaException(String message) e BibliotecaException(String message, Throwable cause”

Código fonte:



```
src/br/com/biblioteca/exception/BibliotecaException.java

... @@ -0,0 +1,12 @@
1 + package br.com.biblioteca.exception;
2 +
3 + public class BibliotecaException extends Exception {
4 +
5 +     public BibliotecaException(String message) {
6 +         super(message);
7 +     }
8 +
9 +     public BibliotecaException(String message, Throwable cause) {
10 +         super(message, cause);
11 +     }
12 + }
```

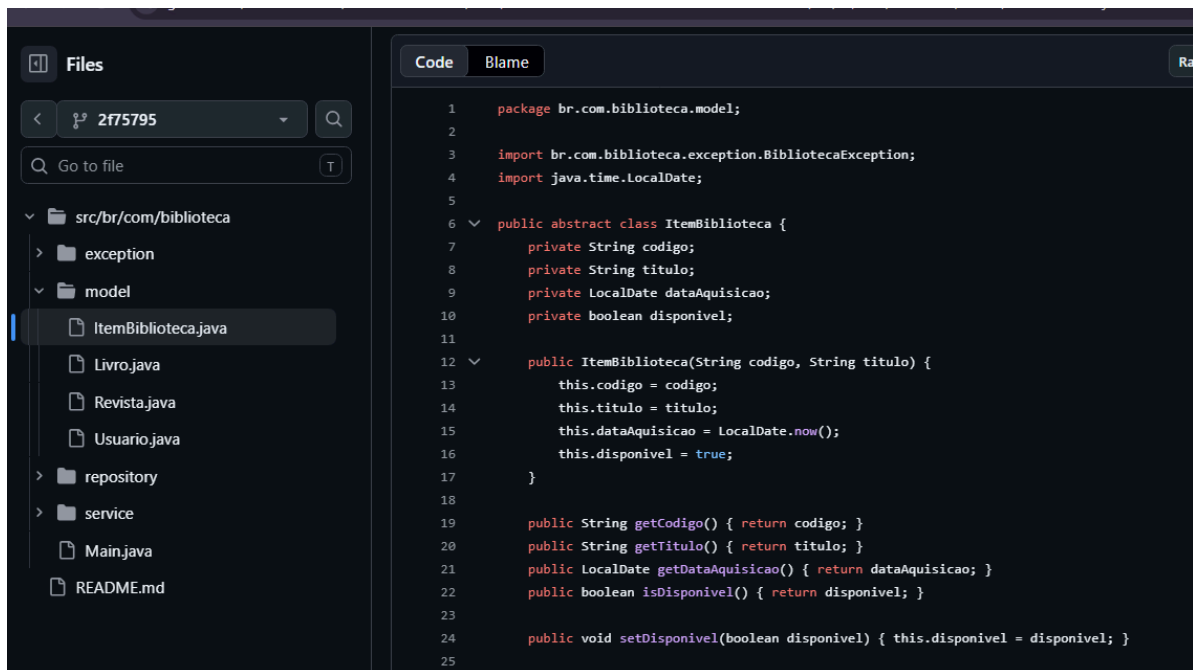
☐ Item Biblioteca

Responsabilidade: Representar um item genérico da biblioteca, armazenando informações comuns e controlando operações de empréstimo e devolução.

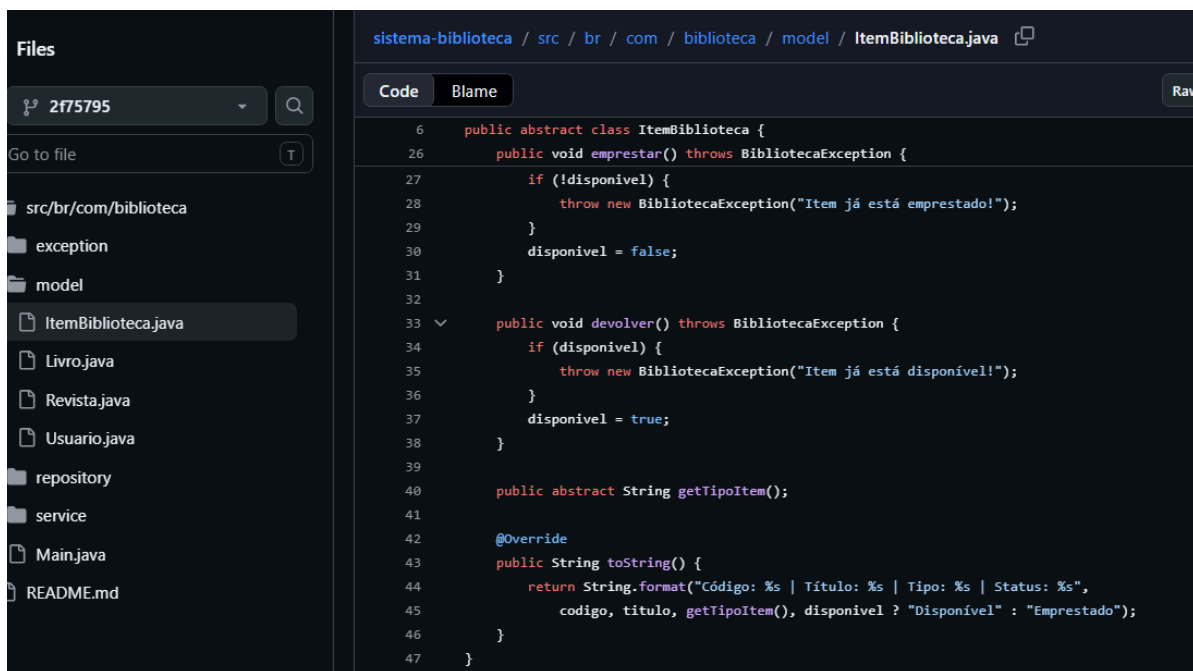
Atributos: Codigo, titulo, dataAquisicao, disponivel

Métodos: Construtor, getters, setter, emprestar(), devolver()

Código fonte:



```
1 package br.com.biblioteca.model;
2
3 import br.com.biblioteca.exception.BibliotecaException;
4 import java.time.LocalDate;
5
6 public abstract class ItemBiblioteca {
7     private String codigo;
8     private String titulo;
9     private LocalDate dataAquisicao;
10    private boolean disponivel;
11
12    public ItemBiblioteca(String codigo, String titulo) {
13        this.codigo = codigo;
14        this.titulo = titulo;
15        this.dataAquisicao = LocalDate.now();
16        this.disponivel = true;
17    }
18
19    public String getCodigo() { return codigo; }
20    public String getTitulo() { return titulo; }
21    public LocalDate getDataAquisicao() { return dataAquisicao; }
22    public boolean isDisponivel() { return disponivel; }
23
24    public void setDisponivel(boolean disponivel) { this.disponivel = disponivel; }
25 }
```



```
sistema-biblioteca / src / br / com / biblioteca / model / ItemBiblioteca.java
Code Blame Raw
6 public abstract class ItemBiblioteca {
26     public void emprestar() throws BibliotecaException {
27         if (!disponivel) {
28             throw new BibliotecaException("Item já está emprestado!");
29         }
30         disponivel = false;
31     }
32
33     public void devolver() throws BibliotecaException {
34         if (disponivel) {
35             throw new BibliotecaException("Item já está disponível!");
36         }
37         disponivel = true;
38     }
39
40     public abstract String getTipoItem();
41
42     @Override
43     public String toString() {
44         return String.format("Código: %s | Título: %s | Tipo: %s | Status: %s",
45             codigo, titulo, getTipoItem(), disponivel ? "Disponível" : "Emprestado");
46     }
47 }
```

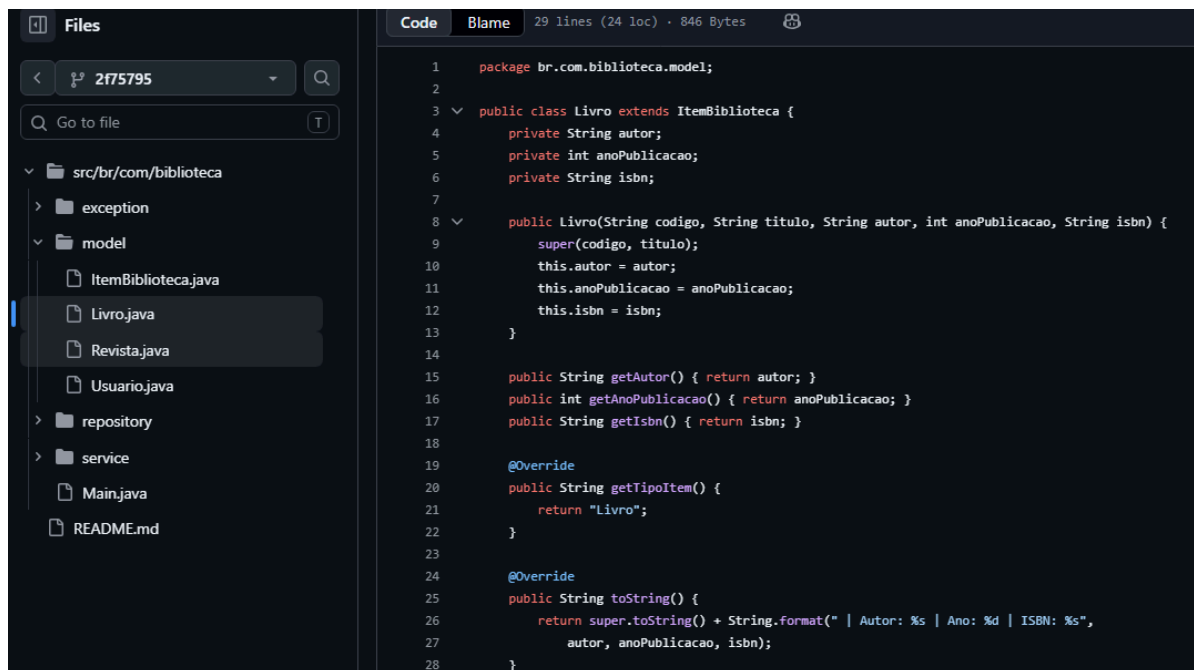
☐ Livro Java

Responsabilidade: Representar os livros da biblioteca, armazenando informações como autor, ano de publicação e ISBN. Também define que o item é do tipo "Livro".

Atributos: São os campos declarados dentro da classe: private String autor; private int anoPublicacao; private String isbn.

Métodos: construtor; getters (autor, anoPublicacao, isbn); getTipoItem(); toString()

Código fonte:



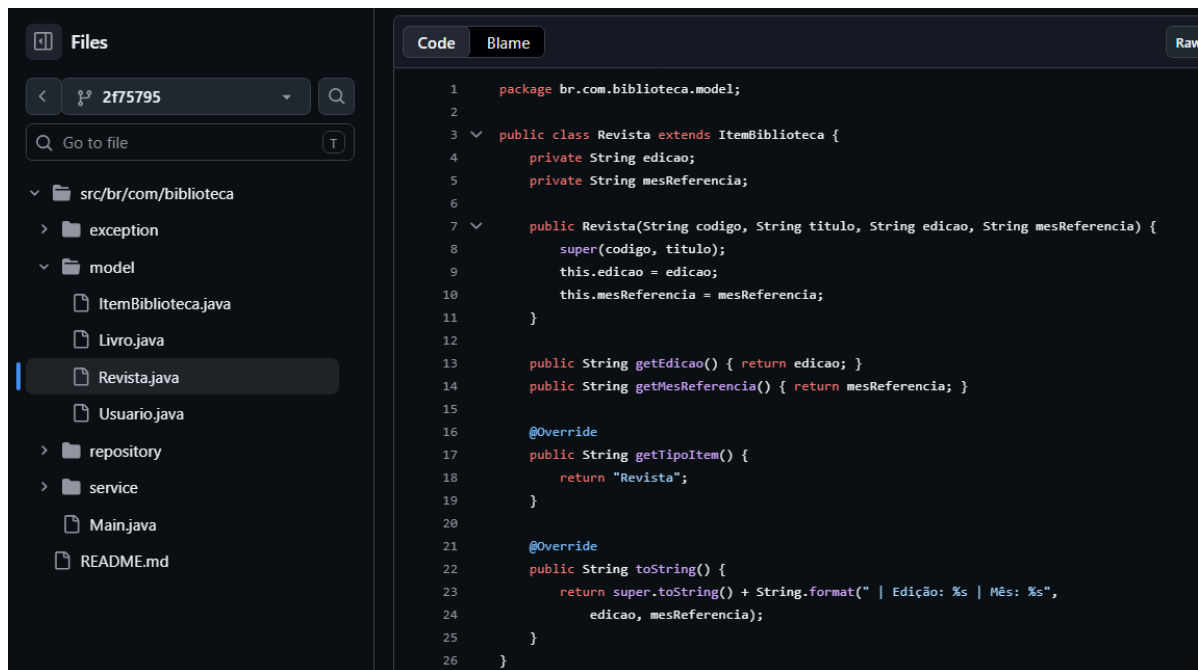
☐ Revista.java

Responsabilidade: Representar as revistas da biblioteca, guardando dados como edição e mês de referência. Também identifica o item como uma revista.

Atributos: private String edicao; private String mesReferencia;

Métodos: public Revista(String codigo, String titulo, String edicao, String mesReferencia); public String getEdicao(); public String getMesReferencia(); public String getTipoItem(); public String toString().

Código fonte:



☐ Usuário.java

Responsabilidade: Representar os usuários cadastrados na biblioteca, armazenando seus dados (id, nome e e-mail) e o histórico de empréstimos realizados.

Atributos: private String id; private String nome; private String email; private List<String> historicoEmprestimos;

Métodos: public Usuario(String id, String nome, String email); public String getId(); public String getNome(); public String getEmail(); public List<String> getHistoricoEmprestimos().

Código fonte:

```
1 package br.com.biblioteca.model;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class Usuario {
7     private String id;
8     private String nome;
9     private String email;
10    private List<String> historicoEmprestimos;
11
12    public Usuario(String id, String nome, String email) {
13        this.id = id;
14        this.nome = nome;
15        this.email = email;
16        this.historicoEmprestimos = new ArrayList<>();
17    }
18
19    public String getId() { return id; }
20    public String getNome() { return nome; }
21    public String getEmail() { return email; }
22    public List<String> getHistoricoEmprestimos() { return historicoEmprestimos; }
23
24    public void adicionarAoHistorico(String codigoItem) {
25        historicoEmprestimos.add(codigoItem);
26    }
27
28    @Override
29    public String toString() {
30        return String.format("ID: %s | Nome: %s | Email: %s", id, nome, email);
31    }
32 }
```

☐ Biblioteca Repository

Responsabilidade: Armazenar e gerenciar os dados da biblioteca, como usuários e itens. Permite adicionar, buscar e listar informações.

Atributos: Ela possui como atributos principais duas listas: na lista de ItemBiblioteca, chamada items, que armazena todos os livros e revistas cadastrados no sistema, e uma lista de Usuario, chamada usuarios, que guarda todos os usuários registrados. Essas listas são inicializadas com ArrayList, o que permite adicionar, remover e percorrer os elementos dinamicamente durante a execução do programa.

Métodos: buscarItemPorCodigo Optional; buscarItensDisponiveis
buscarItensEmprestados; adicionarUsuario; buscarUsuarioPorId;
listarTodosUsuarios.

Código fonte:

```
1 package br.com.biblioteca.repository;
2
3 import br.com.biblioteca.model.ItemBiblioteca;
4 import br.com.biblioteca.model.Usuario;
5 import java.util.ArrayList;
6 import java.util.List;
7 import java.util.Optional;
8 import java.util.stream.Collectors;
9
10 public class BibliotecaRepository {
11     private List<ItemBiblioteca> itens;
12     private List<Usuario> usuarios;
13
14     public BibliotecaRepository() {
15         this.itens = new ArrayList<>();
16         this.usuarios = new ArrayList<>();
17     }
18
19     public void adicionarItem(ItemBiblioteca item) {
20         itens.add(item);
21     }
22
23     public List<ItemBiblioteca> listarTodosItens() {
24         return new ArrayList<>(itens);
25     }
26
27     public Optional<ItemBiblioteca> buscarItemPorCodigo(String codigo) {
28         return itens.stream()
29             .filter(item -> item.getCodigo().equalsIgnoreCase(codigo))
30             .findFirst();
31     }
32 }
```

```
32
33 public List<ItemBiblioteca> buscarItensDisponiveis() {
34     return itens.stream()
35         .filter(ItemBiblioteca::isDisponivel)
36         .collect(Collectors.toList());
37 }
38
39 public List<ItemBiblioteca> buscarItensEmprestados() {
40     return itens.stream()
41         .filter(item -> !item.isDisponivel())
42         .collect(Collectors.toList());
43 }
44
45 public void adicionarUsuario(Usuario usuario) {
46     usuarios.add(usuario);
47 }
48
49 public Optional<Usuario> buscarUsuarioPorId(String id) {
50     return usuarios.stream()
51         .filter(usuario -> usuario.getId().equalsIgnoreCase(id))
52         .findFirst();
53 }
54
55 public List<Usuario> listarTodosUsuarios() {
56     return new ArrayList<>(usuarios);
57 }
58 }
```

☐ Biblioteca Service

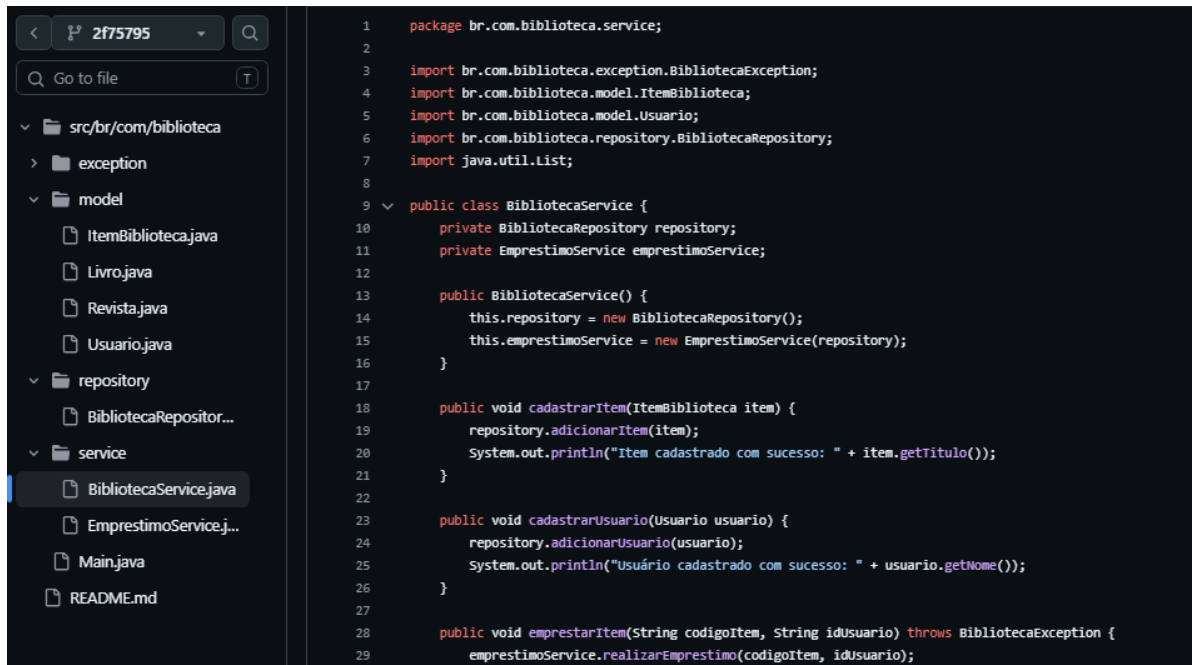
Responsabilidade: Controlar as principais funções da biblioteca, como cadastrar usuários, cadastrar itens, realizar empréstimos, devoluções e exibir listas de itens.

Atributo: repository → acesso aos dados (itens e usuários)

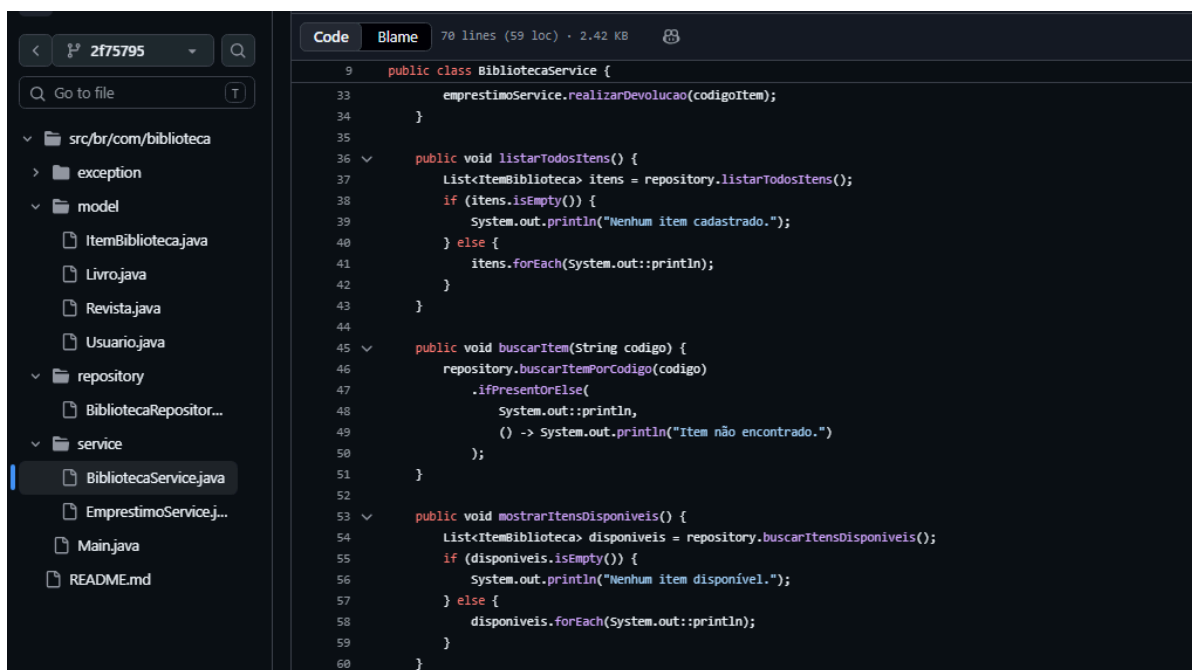
emprestimoService → controla empréstimos/devoluções

Métodos: cadastrarUsuario; emprestarItem; devolverItem; listarTodosItens; buscarItem; mostrarItensDisponiveis; mostrarItensEmprestados.

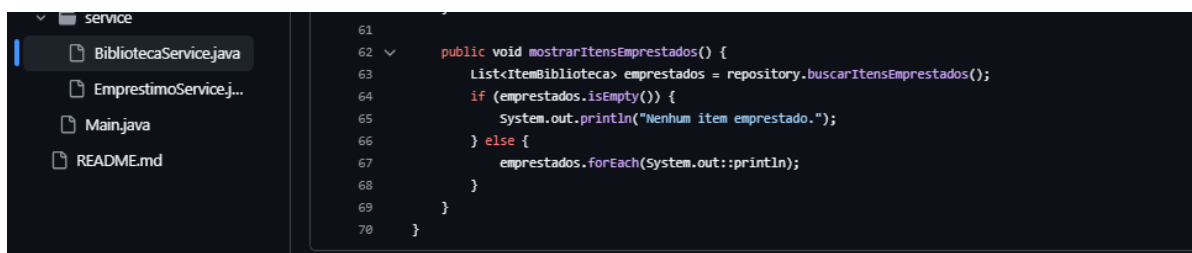
Código fonte:



```
1 package br.com.biblioteca.service;
2
3 import br.com.biblioteca.exception.BibliotecaException;
4 import br.com.biblioteca.model.ItemBiblioteca;
5 import br.com.biblioteca.model.Usuario;
6 import br.com.biblioteca.repository.BibliotecaRepository;
7 import java.util.List;
8
9 public class BibliotecaService {
10     private BibliotecaRepository repository;
11     private EmprestimoService emprestimoService;
12
13     public BibliotecaService() {
14         this.repository = new BibliotecaRepository();
15         this.emprestimoService = new EmprestimoService(repository);
16     }
17
18     public void cadastrarItem(ItemBiblioteca item) {
19         repository.adicionarItem(item);
20         System.out.println("Item cadastrado com sucesso: " + item.getTitulo());
21     }
22
23     public void cadastrarUsuario(Usuario usuario) {
24         repository.adicionarUsuario(usuario);
25         System.out.println("Usuário cadastrado com sucesso: " + usuario.getNome());
26     }
27
28     public void emprestarItem(String codigoItem, String idUsuario) throws BibliotecaException {
29         emprestimoService.realizarEmprestimo(codigoItem, idUsuario);
```



```
33     emprestimoService.realizarDevolucao(codigoItem);
34 }
35
36 public void listarTodosItens() {
37     List<ItemBiblioteca> itens = repository.listarTodosItens();
38     if (itens.isEmpty()) {
39         System.out.println("Nenhum item cadastrado.");
40     } else {
41         itens.forEach(System.out::println);
42     }
43 }
44
45 public void buscarItem(String codigo) {
46     repository.buscarItemPorCodigo(codigo)
47         .ifPresentOrElse(
48             () -> System.out.println("Item não encontrado.")
49         );
50 }
51
52
53 public void mostrarItensDisponiveis() {
54     List<ItemBiblioteca> disponiveis = repository.buscarItensDisponiveis();
55     if (disponiveis.isEmpty()) {
56         System.out.println("Nenhum item disponível.");
57     } else {
58         disponiveis.forEach(System.out::println);
59     }
60 }
```



```
61
62 public void mostrarItensEmprestados() {
63     List<ItemBiblioteca> emprestados = repository.buscarItensEmprestados();
64     if (emprestados.isEmpty()) {
65         System.out.println("Nenhum item emprestado.");
66     } else {
67         emprestados.forEach(System.out::println);
68     }
69 }
70 }
```

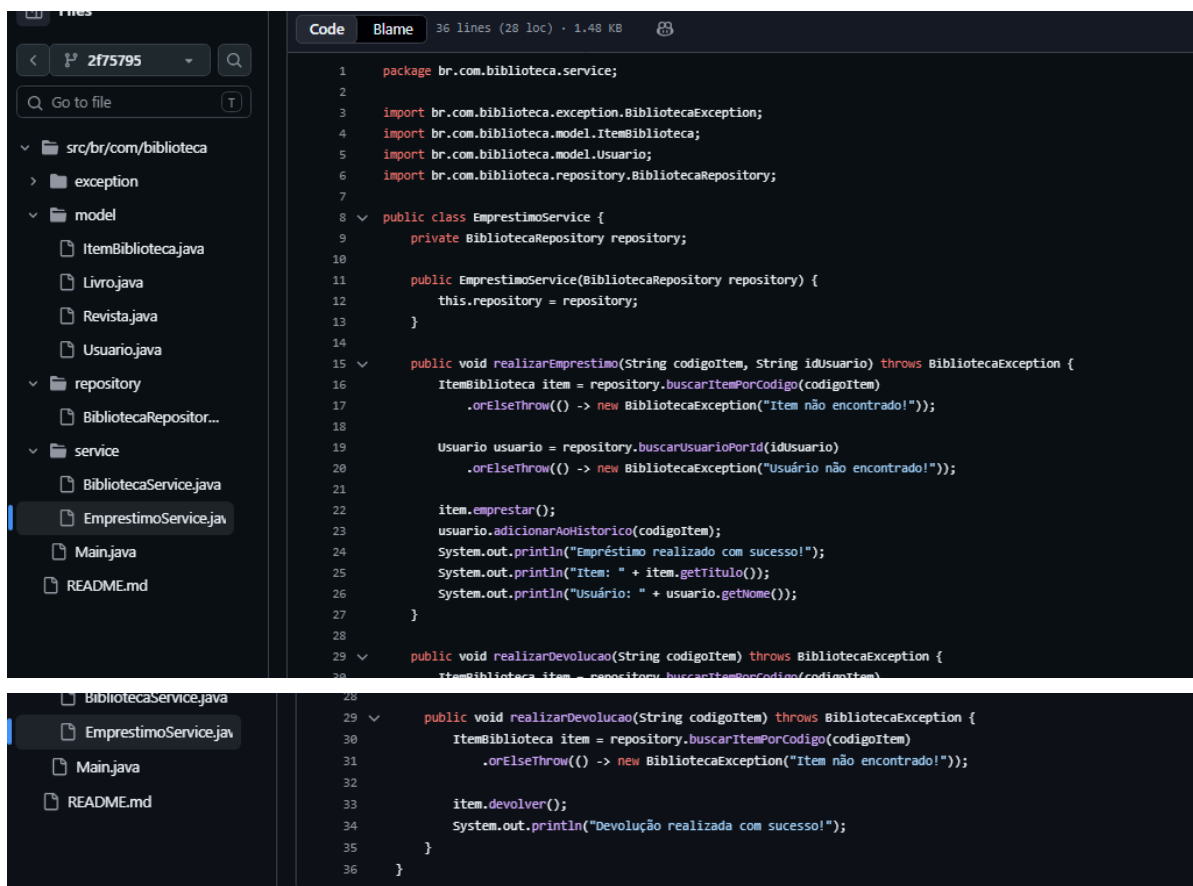
☐ Empréstimo Service

Responsabilidade: Gerenciar os empréstimos e devoluções dos itens da biblioteca, verificando se o item e o usuário existem e atualizando o status dos empréstimos.

Atributos: BibliotecaRepository repository;

Métodos: realizarEmprestimo(String codigoItem, String idUsuario) throws BibliotecaException; realizarDevolucao(String codigoItem).

Código fonte:



```
1 package br.com.biblioteca.service;
2
3 import br.com.biblioteca.exception.BibliotecaException;
4 import br.com.biblioteca.model.ItemBiblioteca;
5 import br.com.biblioteca.model.Usuario;
6 import br.com.biblioteca.repository.BibliotecaRepository;
7
8 public class EmpréstimoService {
9     private BibliotecaRepository repository;
10
11     public EmpréstimoService(BibliotecaRepository repository) {
12         this.repository = repository;
13     }
14
15     public void realizarEmprestimo(String codigoItem, String idUsuario) throws BibliotecaException {
16         ItemBiblioteca item = repository.buscarItemPorCodigo(codigoItem)
17             .orElseThrow(() -> new BibliotecaException("Item não encontrado!"));
18
19         Usuario usuario = repository.buscarUsuarioPorId(idUsuario)
20             .orElseThrow(() -> new BibliotecaException("Usuário não encontrado!"));
21
22         item.emprestar();
23         usuario.adicionarAoHistorico(codigoItem);
24         System.out.println("Empréstimo realizado com sucesso!");
25         System.out.println("Item: " + item.getTitulo());
26         System.out.println("Usuário: " + usuario.getNome());
27     }
28
29     public void realizarDevolucao(String codigoItem) throws BibliotecaException {
30         ItemBiblioteca item = repository.buscarItemPorCodigo(codigoItem)
31             .orElseThrow(() -> new BibliotecaException("Item não encontrado!"));
32
33         item.devolver();
34         System.out.println("Devolução realizada com sucesso!");
35     }
36 }
```

COMPROVAÇÃO DO CÓDIGO FONTE

Teste 1

```

  Digite o número da sua opção: 2

--- Cadastro de Usuário ---
ID do usuário: 1234
Nome completo: Joao Cavalcante
Email: Joao@gmail.com
Usuário cadastrado com sucesso: Joao Cavalcante

SISTEMA DE BIBLIOTECA
1. Cadastrar Item (Livro/Revista)
2. Cadastrar Usuário

```

Teste 2

```

Tipo de Item: 1
Código: 1020
Título: Chagas
Autor: Bruno Partez
Ano de publicação: 1870
ISBN: 1020
Item cadastrado com sucesso: Chagas

```

Teste 3

```

  Digite o número da sua opção: 3
Código: 1001 | Título: Clean Code | Tipo: Livro | Status: Disponível | Autor: Robert Martin | Ano: 2008 | ISBN: 9780132150884
Código: 1002 | Título: Design Patterns | Tipo: Livro | Status: Disponível | Autor: Erich Gamma | Ano: 1994 | ISBN: 9780201633610
Código: 8001 | Título: Veja | Tipo: Revista | Status: Disponível | Edição: Ed. 2756 | Mês: Janeiro/2024
Código: 1020 | Título: Chagas | Tipo: Livro | Status: Disponível | Autor: Bruno Partez | Ano: 1870 | ISBN: 1020

```

Teste 4

```

--- Empréstimo ---
Código do item: 1020
ID do usuário: 1234
Empréstimo realizado com sucesso!
Item: Chagas
Usuário: Joao Cavalcante

```

Teste 5

```

  Digite o número da sua opção: 7
Código: 1001 | Título: Clean Code | Tipo: Livro | Status: Disponível | Autor: Robert Martin | Ano: 2008 | ISBN: 9780132150884
Código: 1002 | Título: Design Patterns | Tipo: Livro | Status: Disponível | Autor: Erich Gamma | Ano: 1994 | ISBN: 9780201633610
Código: 8001 | Título: Veja | Tipo: Revista | Status: Disponível | Edição: Ed. 2756 | Mês: Janeiro/2024

```

Teste 6

```

  9. Sair

  Digite o número da sua opção: 8
Cód.º: 1020 | Título: Chagas | Status: Empréstado | Autor: Bruno Partez | Ano: 1870 | ISBN: 1020

```

Teste 7

```

--- Devolução ---
Código do item: 1020
Devolução realizada com sucesso!

```

Teste 8

```

  Digite o número da sua opção: 4

  Digite o código do item: 1020
Código: 1020 | Título: Chagas | Tipo: Livro | Status: Disponível | Autor: Bruno Partez | Ano: 1870 | ISBN: 1020

```

Teste 9

```

  6. Devolver Item
  7. Mostrar Items Disponíveis
  8. Mostrar Items Empréstados
  9. Sair

  Digite o número da sua opção: 9
Saindo do sistema...
PS C:\Users\Brazo>

```

- **DIFICULDADES ENCONTRADAS**

A equipe apresentou dificuldades em organizar o código fonte em classes.

- **APRENDIZADOS OBTIDOS**

A equipe ao todo aprendeu como organizar e estudar o código apresentado neste trabalho, além de aprender como se organizar e a trabalhar em equipe.

CONCLUSÃO

O projeto do sistema de biblioteca foi desenvolvido com o objetivo de simular o funcionamento de uma biblioteca de forma simples e organizada, aplicando conceitos importantes de Programação Orientada a Objetos (POO), como classes, herança, encapsulamento e polimorfismo.

Durante a implementação, foi possível estruturar o sistema em camadas bem definidas, separando responsabilidades entre modelos, repositório e serviços. Os itens da biblioteca, como livros e revistas, foram representados por classes específicas que herdam de uma classe base, permitindo reutilização de código e maior organização. Além disso, a classe de usuários possibilita o controle de cadastro e histórico de empréstimos.

Portanto, o projeto contribuiu para o aprendizado prático de conceitos fundamentais da programação orientada a objetos, além de demonstrar como estruturar um sistema de forma modular, facilitando a manutenção, entendimento e futuras melhorias.

MANUAL DO USUÁRIO

O Sistema de Biblioteca é um programa em Java que permite gerenciar livros de uma biblioteca. Com ele é possível cadastrar livros, listar acervo, buscar por código, além de controlar empréstimos e devoluções.

- **Como usar o sistema**

Ao iniciar o programa, será exibido um menu:

--- **Sistema de Biblioteca** ---

1. Cadastrar livro
2. Listar livros
3. Buscar livro pelo código
4. Emprestar livro
5. Devolver livro
6. Mostrar livros disponíveis

7. Mostrar livros emprestados

8. Sair

O usuário deve digitar o número da opção desejada

- **Requisitos para execução**

- 1) Java JDK 8 ou superior instalado
- 2) Ambiente de execução (VS Code, IntelliJ ou terminal)
- 3) Nenhuma biblioteca externa é necessária (usa apenas ArrayList e Scanner)

- **Cadastro de livros**

Ao escolher a opção 1, o sistema solicita:

Título do livro

Autor

Ano de publicação

Código único do livro

☐ **Exemplo:**

Título: Dom Casmurro

Autor: Machado de Assis

Ano: 1899

Código: L001

- **Listagem de livros**

A opção 2 mostra todos os livros cadastrados, incluindo:

Código

Título

Autor

Ano

Status (Disponível ou Emprestado)

- **Busca de livro**

A opção 3 permite buscar um livro pelo código.

☐ **Exemplo:**

Digite o código: L001

Se encontrado, o sistema exibe os dados completos do livro.

- **Empréstimo de livros**

A opção 4 altera o status do livro para Emprestado. Só funciona se o livro estiver disponível. Caso contrário, o sistema informa que já foi emprestado

- **Devolução de livros**

A opção 5 devolve o livro ao estado Disponível. Só funciona se o livro estiver emprestado

- **Livros disponíveis**

Mostra apenas os livros com status:

Disponível.

- **Livros emprestados**

Mostra apenas os livros com status:

Emprestado.

- **Possíveis mensagens do sistema**

"Livro cadastrado com sucesso!"

"Livro emprestado com sucesso!"

"O livro já está emprestado!"

"Livro devolvido com sucesso!"

"Livro não encontrado."

"Nenhum livro cadastrado."